M2S-06: Step 6 — Integrate: Reconnect Integrations

> **Series:** M2S — Managed to SaaS Migration | **Notebook:** 6 of 9 | **Phase:** Upgrade | **Step:** Integrate | **Created:** March 2026 | **Last Updated:** 11/11/2026

With OneAgents reporting to SaaS and configurations migrated, the next challenge is ensuring every external system that depends on Dynatrace is reconnected. Dashboards need updated links, alerting channels need validation, CI/CD pipelines need new API endpoints, and ITSM integrations need reconfiguration. This notebook provides a systematic approach to reconnecting every integration point.

> **M2S Migration Journey — 3 Phases / 9 Steps**
>
> **Plan:** 1. Discover | 2. Strategize | 3. Design
>
> **Upgrade:** 4. Prepare | 5. Execute | **6. Integrate**
>
> **Run:** 7. Enable | 8. Expand | 9. Optimize

Table of Contents

1. Dashboard Integration
2. Alerting and Notification Integration
3. CI/CD Pipeline Integration
4. ITSM and Incident Management
5. Extension Migration
6. API Script Migration
7. Synthetic Monitor Migration
8. Step Completion Checklist

Prerequisites

Requirement	Details
Steps 1-5 Complete	Discovery, strategy, design, preparation, and execution phases finished
SaaS Tenant Active	OneAgents reporting data to SaaS environment
SaaS API Tokens	Tokens with appropriate scopes for each integration
Integration Inventory	List of all third-party integrations from Step 1 (Discover)
Access to External Systems	Admin access to Slack, Teams, PagerDuty, ServiceNow, CI/CD tools

> **Order of Operations — You Are Here**
>
> This notebook covers operations **7-8** of the 11-step Order of Operations (see **M2S-02: Step 2 — Strategize**) defined in Step 2:

#	Operation	Status
1	Assess — Inventory current environment	Step 4: Prepare ✅
2	Provision — SaaS tenant and access (SSO)	Step 4: Prepare ✅
3	Install — New ActiveGates in parallel	Step 4: Prepare ✅
4	Migrate — Configuration and integrations	Step 5: Execute ✅
5	Rebuild — Dashboards, zones, alerts	Step 5: Execute ✅
6	Redirect — OneAgents to SaaS	Step 5: Execute ✅
7	**Reconnect — Integrations and extensions**	**This notebook**
8	**Migrate — Remaining config and integrations**	**This notebook**
9	Validate — Data flow and performance	Step 9: Optimize
10	Cutover — Full switch to SaaS	Step 9: Optimize
11	Decommission — Managed environment	Step 9: Optimize

1. Dashboard Integration

Dashboards migrated via the SaaS Upgrade Assistant retain their structure, but several elements need manual attention. Internal links, entity references, and ownership must be validated against the new SaaS environment.

What Needs Updating

Item	Why	Action
Dashboard ownership	Managed user IDs differ from SaaS user IDs	Reassign ownership via SaaS Upgrade Assistant bulk edit or manually
Internal links	URLs pointing to Managed UI pages are broken	Update all `https://{managed-url}/e/{env-id}/...` links to the SaaS tenant (see *URL Format Changes* below)
Management zone filters	MZ IDs may differ between environments	Verify MZ filters resolve correctly; update IDs if needed
Hardcoded entity IDs	Entity IDs are environment-scoped — the SaaS tenant mints new ones	Re-point each reference; prefer tag or management-zone filters over fixed IDs where the tile supports them
Tile data sources	Some tiles may reference unavailable data	Verify every tile renders data

Dashboard Validation Strategy

Work through dashboards in priority order:

1. **Executive dashboards** — High visibility, first to be noticed if broken
2. **Operations dashboards** — Used for daily monitoring decisions
3. **Team-specific dashboards** — Application and service team views
4. **Report dashboards** — Used for periodic reporting

Verify Dashboards Are Present

Dashboards are **Document Service** objects on SaaS, not Grail entities — `fetch dt.entity.dashboard` is **not valid DQL** and returns no data. To list and count dashboards on your SaaS tenant so you can compare against your Managed inventory, use the **Document Service API**. It is a platform API: authenticate with a **platform token** (`Authorization: Bearer …`) that carries the `document:documents:read` permission, not a classic API token.

```
# Count dashboards on SaaS (compare to your Managed inventory)
curl -s "https://{tenant}.apps.dynatrace.com/platform/document/v1/documents?filter=type='dashboard'" \
  -H "Authorization: Bearer {PLATFORM_TOKEN}" | jq '.totalCount'

# List dashboard names and owners
curl -s "https://{tenant}.apps.dynatrace.com/platform/document/v1/documents?filter=type='dashboard'" \
  -H "Authorization: Bearer {PLATFORM_TOKEN}" | jq -r '.documents[] | "\(.name)\t\(.owner)"'
```

This lists dashboards built on the new Dashboards app. Classic dashboards migrated by the Upgrade Assistant are listed in the classic **Dashboards (classic)** page and the classic Dashboards API (`/api/config/v1/dashboards` , `ReadConfig` scope). Alternatively, open each app in the SaaS UI and compare against your Managed dashboard inventory.

> **Sources:** Platform tokens (DT docs) — *"Platform tokens are directly usable with the APIs offered by the Dynatrace platform services."*

URL Format Changes

Every internal link in dashboards must be updated from Managed format to SaaS format. For **classic** pages, the path after the environment is unchanged — only the host part moves:

Page	Managed URL	SaaS URL
Classic pages (hosts, services, classic dashboards)	`https://{managed}/e/{env-id}/#<page>;id=<ID>`	`https://{tenant-id}.live.dynatrace.com/#<page>;id=<NEW-ID>`
New platform apps (Dashboards, Notebooks, Services, Infrastructure & Operations)	N/A (Managed)	Copy the link from the app's share or address bar on `https://{tenant-id}.apps.dynatrace.com/ui/apps/…` — do not hand-build these paths

Remember that the `id=` part changes too: entity and dashboard IDs are minted by the new tenant (FAQ-25 § 4), so a link with the host rewritten but the old ID still points nowhere.

> **Tip:** Use the classic Dashboards API to search for and replace Managed URLs across all classic dashboards. This is faster and more reliable than manual editing for large dashboard inventories.

Replacing Hardcoded Entity IDs

Where a classic tile is pinned to a fixed entity ID, re-point it to the new ID, or — where the tile offers a tag or management-zone filter — switch to that filter so the tile survives the next ID change.

2. Alerting and Notification Integration

Alerting is the most critical integration to validate. A missed alert during migration can mean an undetected production incident. Every notification channel must be tested end-to-end before declaring the migration complete.

What Changes Between Managed and SaaS

Component	Managed	SaaS
Problem notifications	Classic notification rules	Migrate to SaaS notification rules or Workflow triggers
Webhook source URL	`https://{managed}/e/{env-id}/...`	`https://{tenant}.apps.dynatrace.com/...`
Alerting profiles	Migrated via SaaS Upgrade Assistant	Verify filters and thresholds match
Notification templates	May reference Managed-specific variables	Update template variables for SaaS

Integration Endpoints to Update

Integration	What Changes	What Stays the Same
Slack	Dynatrace source URLs in message payloads	Webhook URL
Microsoft Teams	Dynatrace source URLs in adaptive cards	Webhook URL
PagerDuty	Create new integration in SaaS; Dynatrace source URLs	Routing key
ServiceNow	Create new Dynatrace integration user in SNOW	ServiceNow instance URL
Custom webhooks	All Dynatrace API endpoints in webhook payloads	External system endpoints
Email	Notification sender address (Dynatrace SaaS domain)	Recipient distribution lists

Converting to Workflow-Based Notifications

SaaS provides the AutomationEngine for advanced alerting workflows. Consider converting Managed problem notifications to Workflow triggers for:

- **Conditional routing** — Route alerts based on management zone, severity, or tags
- **Enrichment** — Add entity details, runbook links, or Dynatrace Intelligence context to notifications
- **Multi-channel delivery** — Send to Slack AND PagerDuty from a single workflow
- **Auto-remediation** — Trigger runbooks or API calls in response to problems

End-to-End Alert Validation

For each notification channel:

1. **Configure** — Set up the notification integration in SaaS
2. **Trigger** — Create a test problem or use a custom event to fire an alert
3. **Verify delivery** — Confirm the alert arrives at the destination
4. **Check content** — Verify URLs, entity names, and context are correct
5. **Test resolution** — Close the problem and verify the resolution notification

> **Warning:** Do not skip testing resolution notifications. Many integrations (PagerDuty, ServiceNow) auto-resolve incidents when the Dynatrace problem closes. If resolution notifications are broken, incidents pile up in your ITSM system.

Verify Alert Delivery Using Detected Problems

After configuring notifications, monitor detected problems to confirm alerts are flowing:

```
// Recent problems — verify these triggered notifications in your external systems
fetch dt.davis.problems, from:-24h
| fieldsKeep display_id, event.name, event.status, event.category, timestamp
| sort timestamp desc
| limit 20
```

```
// Problem trend over last 7 days — confirms Dynatrace Intelligence baseline is building
fetch dt.davis.problems, from:-7d
| makeTimeseries count = count(default: 0), interval: 24h
```

3. CI/CD Pipeline Integration

Every CI/CD pipeline that integrates with Dynatrace needs its connection parameters updated. This includes deployment event notifications, quality gate evaluations, and release tracking.

URL Format Change

All Dynatrace API calls must switch from Managed format to SaaS format:

API	Managed Format	SaaS Format	Survives the Gen3 upgrade?
Environment API v2	`https://{managed}/e/{env-id}/api/v2/...`	`https://{tenant}.live.dynatrace.com/api/v2/...`	Ingest paths yes; several read/config paths **no** — see below
Configuration API v1	`https://{managed}/e/{env-id}/api/config/v1/...`	`https://{tenant}.live.dynatrace.com/api/config/v1/...`	**Mostly no**
Platform API	N/A (Managed)	`https://{tenant}.apps.dynatrace.com/platform/...`	Yes

> **Repoint once, not twice.** This step is a host-and-path rewrite for the Managed → SaaS move, and on that axis the table above is correct. But a separate change is coming: when the tenant later upgrades to the latest Dynatrace, **most of `/api/config/v1` and a number of `/api/v2` read and configuration paths stop answering**, while the **ingest** endpoints (`/api/v2/logs/ingest` , `/metrics/ingest` , `/events/ingest` , `/api/v2/otlp` , `/api/bizevents/ingest`) and everything under `/platform/` carry forward.
>
> The narrow `/api/config/v1` carve-outs that do survive are worth knowing, because they are easy to confuse with neighbours that do not: `/service/requestAttributes` , `/service/requestNaming` and `/service/customServices` survive, while `/conditionalNaming/*` and `/calculatedMetrics/service` do not — and `/calculatedMetrics/mobile` survives while `/calculatedMetrics/service` does not.
>
> If a script is being rewritten anyway, check it against AUTOM-02's catalog now and move it to its Gen3 equivalent in the same pass rather than migrating it twice. See the Classic → Gen3 doorway in the `-START-HERE-` playbook for the sequence.

CI/CD Tools to Update

Tool	Configuration to Update	Where to Change
Jenkins	Dynatrace plugin configuration, API token	Jenkins system configuration and pipeline credentials
GitHub Actions	Repository secrets: `DYNATRACE_URL`, `DYNATRACE_API_TOKEN`	Repository Settings > Secrets and Variables
Azure DevOps	Service connections, pipeline variables	Project Settings > Service Connections
GitLab CI	CI/CD variables: `DT_URL`, `DT_TOKEN`	Settings > CI/CD > Variables
Ansible	Dynatrace collection connection variables	Inventory or playbook variables
Terraform	Provider configuration (`environment_url`, `api_token`)	Provider block in `.tf` files or environment variables

Generate New API Tokens

Create dedicated API tokens for each CI/CD integration. Follow the principle of least privilege:

Integration	Required Token Scopes
Deployment events	`events.ingest`
Configuration push	`settings.write`, `settings.read`
Entity query	`entities.read`
Metric ingestion	`metrics.ingest`
SLO evaluation	`slo.read`
Full automation	Combine scopes per tool requirements

> **Important:** Never reuse a single token across all CI/CD tools. If one token is compromised, all integrations are affected. Create one token per tool with only the scopes that tool requires.

Deployment Event Validation

After updating CI/CD configurations, trigger a test deployment and verify that deployment events appear in SaaS.

Deployment events are identified by **`event.type == "CUSTOM_DEPLOYMENT"`**. `CUSTOM_DEPLOYMENT` is an event *type*; filtering on `event.kind` instead never matches, and returns an empty result that looks exactly like a pipeline that sent nothing. The two queries below were executed successfully against a live tenant, but that tenant held no deployment events, so they were confirmed to run, not confirmed to return rows. The field names come from the documentation and the semantic dictionary. Your first real test deployment is the positive check.

> **Sources:** Davis events model (DT docs) — `event.type`: *"The unique type identifier of a given event. Must be one of:"* (the list includes `CUSTOM_DEPLOYMENT`). **Dictionary:** `event.kind` (`stable`), `deployment.release_version` (`experimental`), `deployment.release_stage` (`experimental`); no row for `event.deployment.*`, read 09/28/2026 (control: the same filter returned `deployment.release_*` rows).

```
// Verify deployment events are being received from CI/CD pipelines
fetch events, from:-24h
| filter event.type == "CUSTOM_DEPLOYMENT"
| fieldsKeep event.type, event.name, dt.entity.service, timestamp
| sort timestamp desc
| limit 10
```

Release Tracking

If you use Dynatrace release tracking, verify that version metadata is being captured correctly after the CI/CD update. The release fields are `deployment.release_version` and `deployment.release_stage` (both `experimental` in the semantic dictionary) — not `event.deployment.*`, which does not exist:

```
// Check release versions captured by Dynatrace via deployment events
// Release version and stage are carried on the deployment event, not as entity fields
fetch events, from:-7d
| filter event.type == "CUSTOM_DEPLOYMENT"
| fieldsKeep event.name, dt.entity.service, deployment.release_version, deployment.release_stage, timestamp
| sort timestamp desc
| limit 20

// Note: releaseVersion / releaseStage are not directly addressable fields on dt.entity.service in DQL.
// Release tracking data surfaces through deployment events (event.type == "CUSTOM_DEPLOYMENT") or
// via the Release Monitoring app in the Dynatrace UI.
```

4. ITSM and Incident Management

ITSM integrations are business-critical — they drive incident response workflows. A broken ITSM integration means problems detected by Dynatrace never reach the teams that can fix them.

ServiceNow Integration

Configuration Item	Action
Dynatrace instance URL	Update from Managed URL to `https://{tenant}.live.dynatrace.com`
Integration user	Create new integration user with SaaS API token
Problem webhook	Reconfigure webhook to send from SaaS
CMDB synchronization	Update API endpoint for entity data push
Event management	Verify event forwarding from SaaS to ServiceNow event management

Jira Integration

Configuration Item	Action
Webhook endpoint	Update Dynatrace webhook to call Jira from SaaS
Problem-to-ticket mapping	Verify field mappings work with SaaS problem format
Back-link URLs	Update Jira ticket templates to link to SaaS URLs

CMDB Synchronization

If you synchronize Dynatrace entities to a CMDB:

1. **Update API endpoint** — Point CMDB sync to SaaS Entities API v2
2. **Update authentication** — New SaaS API token with `entities.read` scope
3. **Verify entity mapping** — Entity IDs may differ; use entity selectors or tags for matching
4. **Test sync cycle** — Run a sync and confirm entities appear correctly in the CMDB

Validate ITSM Connectivity

```
// Check for recent closed problems — use these to verify ITSM tickets were created and resolved
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| fieldsKeep display_id, event.name, event.category, timestamp
| sort timestamp desc
| limit 10
```

5. Extension Migration

Extensions provide monitoring for technologies not covered by OneAgent auto-instrumentation. The extension framework has changed significantly between Managed and SaaS.

Extension Framework Comparison

Framework	Managed	SaaS	Action
Extensions 1.0 (EF1)	Reached end of support 2025-03-31; the JMX/PMI EF1 deprecation notice no longer applies to Managed (Managed 1.348)	Reached end of support 2025-03-31 (JMX/PMI: July 1, 2027)	Rebuild as Extensions 2.0
Extensions 2.0 (EF2)	Supported	Fully supported	Migrate configuration; may need re-signing
ActiveGate extensions	Supported	Supported — plan host-based ActiveGates; SQL extensions can also run on Kubernetes through the Dynatrace Operator	Deploy on host-based ActiveGates in SaaS
Custom plugins (OneAgent / ActiveGate plugins on EF1)	End of support with EF1	Not supported	Rebuild as Extensions 2.0

Extensions 2.0 Requirements in SaaS

Requirement	Details
Host-based ActiveGate	Plan host-based ActiveGates for remote extensions. The documented Kubernetes path is narrower: *"Run SQL monitoring extensions on Kubernetes using Dynatrace Operator."*
Extension signing	Extensions must be signed with a valid certificate for SaaS
Credential Vault	Extension credentials must be re-entered in SaaS Credentials Vault

Requirement	Details
Monitoring configuration	Extension monitoring configurations need recreation in SaaS

> **Sources:** End of support announcements (DT docs) — *"EF1 JMX and PMI extensions reach end of support on July 1, 2027. After this date, they will no longer be supported in SaaS environments."*; Managed 1.348 (DT docs) — *"The deprecation notice for JMX/PMI EF1 no longer applies to Dynatrace Managed and has been removed."*; Extensions (DT docs) — *"Run SQL monitoring extensions on Kubernetes using Dynatrace Operator."*

Cloud Integration Verification

Cloud platform integrations (AWS, Azure, GCP) require reconfiguration in SaaS because credentials are not portable:

Cloud Platform	Configuration to Update
AWS	IAM role or access key for CloudWatch metrics and ActiveGate
Azure	Service principal credentials for Azure Monitor
GCP	Service account key for Google Cloud Monitoring

After reconfiguring cloud integrations, verify metrics are flowing:

```
// Verify AWS cloud metrics are flowing after integration reconfiguration
//
// The `dt.cloud.aws.*` metric namespace DOES NOT EXIST in any spelling (verified 08/12/2026).
// AWS CloudWatch metrics are published as
cloud.aws.&lt;service&gt;.&lt;CloudWatchName&gt;.By.&lt;Dimension&gt; —
// the CloudWatch name keeps its PascalCase and the split dimension is part of the key. A
// timeseries against a missing key returns an EMPTY result rather than an error, so this cell
// silently drew nothing. Enumerate with:
//   metrics | filter startsWith(metric.key, "cloud.aws") | fields metric.key | sort metric.key asc
timeseries cpu = avg(cloud.aws.ec2.CPUUtilization.By.InstanceId), from:-1h, by:{InstanceId}
| fieldsAdd avg_cpu = arrayAvg(cpu)
| fields InstanceId, avg_cpu
| sort avg_cpu desc
| limit 5
```

```
// Verify Azure cloud metrics are flowing
//
// Corrected 08/12/2026: `dt.cloud.azure.vm.cpu_usage` does not exist. The `dt.cloud.azure.*`
// namespace is real but carries only VM *counts* (running / stopped / initializing, by region and
// scale set) — there is no CPU metric in it. Azure Monitor resource metrics are published as
// cloud.azure.&lt;provider_namespace&gt;.&lt;resource_type&gt;.&lt;MetricName&gt;, so VM CPU is
PercentageCPU.
// (For OneAgent-monitored Azure VMs, dt.host.cpu.usage is the host-level equivalent.) Enumerate:
//   metrics | filter startsWith(metric.key, "cloud.azure") | fields metric.key | sort metric.key asc
timeseries cpu = avg(cloud.azure.microsoft_compute.virtualmachines.PercentageCPU), from:-1h
| fieldsAdd avg_cpu = arrayAvg(cpu)
| fields avg_cpu
```

Extension Health Check

```
// Check ActiveGate health — extensions run on host-based ActiveGates carrying the
// EXTENSION_CONTROLLER module, so filter to exactly those rather than listing every AG
smartscapeNodes "ACTIVEGATE"
| filter is_containerized == false and iAny(modules[] == "EXTENSION_CONTROLLER")
| fields name, version = dt.active_gate.version, group = dt.active_gate.group.name, modules
| sort name asc

// Correction (verified 07/2026): this cell previously ran `fetch dt.entity.active_gate` and
// carried a note claiming Smartscape had no ActiveGate node. Both were wrong. There is NO classic
// ActiveGate entity type in any spelling (active_gate, environment_active_gate,
// environment_activegate), so the classic query returned zero rows in every tenant —
// indistinguishable from "no ActiveGates deployed". `smartscapeNodes "ACTIVEGATE"` (no
// underscore) is the working path, and it works on tenants today.
// Field maps: entity.name → name, softwareVersion → dt.active_gate.version,
// networkZone → dt.network_zone.id.
// is_containerized and modules are Smartscape-only fields — the classic entity never exposed them,
// so this host-based-AG filter was not expressible before.
```

6. API Script Migration

Most organizations have scripts and tools that call Dynatrace APIs for automation, reporting, or meta-monitoring. Every one of these must be updated.

URL Format Change

API Endpoint	Managed Format	SaaS Format
Environment API v2	`https://{managed}/e/{env-id}/api/v2/...`	`https://{tenant}.live.dynatrace.com/api/v2/...`
Configuration API v1	`https://{managed}/e/{env-id}/api/config/v1/...`	`https://{tenant}.live.dynatrace.com/api/config/v1/...`
Cluster API	`https://{managed}/api/cluster/v2/...`	N/A — use Account Management API instead

> **Important:** The Cluster Management API does not exist in SaaS. Scripts that use cluster-level APIs (user management, environment provisioning, license management) must be rewritten to use the Account Management API or the SaaS UI.

Common Script Types to Audit

Script Type	Typical Purpose	Update Required
Health check scripts	Monitor Dynatrace cluster availability	Update URL and token; remove cluster-specific checks
Reporting scripts	Pull metrics and SLO data for reports	Update URL and token
Provisioning scripts	Create management zones, tags, dashboards	Update URL and token
Token rotation scripts	Rotate API tokens automatically	Rewrite for SaaS token management
Backup scripts	Export configurations periodically	Update URL and token

Script Type	Typical Purpose	Update Required
Meta-monitoring	Monitor the monitoring platform itself	Update URL and token; adjust health checks for SaaS architecture

Token Migration Strategy

1. **Inventory all tokens** — List every API token used in Managed with its purpose and scopes
2. **Create SaaS equivalents** — Generate new tokens in SaaS with matching (or tighter) scopes
3. **Update scripts** — Replace Managed tokens with SaaS tokens in all scripts and configuration files
4. **Test each script** — Run every script and verify it completes successfully against SaaS
5. **Document the mapping** — Record which SaaS token replaces which Managed token

Meta-Monitoring Considerations

If you monitor Dynatrace itself (meta-monitoring), the health checks change:

Managed Health Check	SaaS Equivalent
Cluster node availability	Not applicable — Dynatrace manages SaaS infrastructure
Cassandra/Elasticsearch health	Not applicable
ActiveGate connectivity to cluster	ActiveGate connectivity to SaaS endpoints
OneAgent connectivity to cluster	OneAgent connectivity to SaaS endpoints
Disk space on cluster nodes	Not applicable
API endpoint availability	API endpoint availability (same concept, new URL)

Verify API Connectivity

Use a simple API call to confirm SaaS API access is working:

```
# Test SaaS API connectivity
curl -s -o /dev/null -w "%{http_code}" \
  "https://{tenant}.live.dynatrace.com/api/v2/entities?pageSize=1" \
  -H "Authorization: Api-Token {SAAS_TOKEN}"
# Expected: 200
```

```
# Verify token scopes
curl -s "https://{tenant}.live.dynatrace.com/api/v2/apiTokens/lookup" \
  -H "Authorization: Api-Token {SAAS_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{"token": "{SAAS_TOKEN}"}' | jq '.scopes'
```

> **Deprecation — Dynatrace API 1.348 (rollout start 10/12/2026, staged tenant rollout).** The API 1.348 changelog marks the whole `/apiTokens` endpoint family deprecated — *"The following endpoints are deprecated"*, covering `GET` and `POST /apiTokens`, `POST /apiTokens/lookup` and the per-token `GET` / `PUT` / `DELETE`. No successor is named. Deprecated endpoints keep working during the deprecation period, so the scope check above (`POST /api/v2/apiTokens/lookup`) and the classic token it inspects remain the working path — re-check the API 1.348 changelog (DT docs) for a successor before building new automation on `/apiTokens/lookup`.

7. Synthetic Monitor Migration

Synthetic monitors cannot be directly migrated from Managed to SaaS. They must be recreated because synthetic execution depends on ActiveGate infrastructure and location IDs that differ between environments.

Migration by Monitor Type

Monitor Type	Migration Approach
HTTP monitors	Export configuration; recreate in SaaS with same URLs and assertions
Browser monitors	Export Selenium/Synthetic scripts; import into new SaaS monitors
Browser clickpath	Re-record or import clickpath scripts

Private Synthetic Locations

If you use private synthetic locations (monitoring from inside your network):

1. **Deploy Synthetic-enabled ActiveGates** in SaaS — these replace your Managed Synthetic ActiveGates
2. **Create new private locations** in SaaS pointing to the new ActiveGates
3. **Assign monitors** to the new private locations
4. **Test execution** — verify monitors run from the new locations

Public Synthetic Locations

Public locations are available in both Managed and SaaS. Simply recreate the monitors and assign the same geographic locations.

Verify Synthetic Monitors Are Executing

```
// Count synthetic monitors in SaaS by type — compare to your Managed inventory
// Browser, HTTP and multi-protocol monitors are separate node types; counting BROWSER_MONITOR alone misses two of them
smartscapeNodes "BROWSER_MONITOR", "HTTP_MONITOR", "NETWORK_AVAILABILITY_MONITOR"
| summarize monitorCount = count(), by:{type}
| sort type asc

// Smartscape (preferred, verified 07/2026): dt.entity.synthetic_test maps to the BROWSER_MONITOR
// node (individual steps are a separate BROWSER_MONITOR_STEP node). HTTP monitors are HTTP_MONITOR,
// multi-protocol monitors NETWORK_AVAILABILITY_MONITOR, private locations SYNTHETIC_LOCATION. This
// corrects an earlier note here that claimed no Smartscape equivalent existed. Unlike ActiveGate,
// `fetch dt.entity.synthetic_test` does still work and remains a genuine fallback — it reads the
// classic entity store, which can retain entities Smartscape (live topology) no longer lists.
// Classic fallback (browser monitors only): fetch dt.entity.synthetic_test | summarize monitorCount = count()
```

```
// List synthetic monitors with their names and types — verify expected monitors are present
smartscapeNodes "BROWSER_MONITOR", "HTTP_MONITOR", "NETWORK_AVAILABILITY_MONITOR"
| fields name, type
| sort type asc, name asc

// Smartscape (preferred, verified 07/2026): dt.entity.synthetic_test maps to the BROWSER_MONITOR
// node (individual steps are a separate BROWSER_MONITOR_STEP node). HTTP monitors are HTTP_MONITOR,
// multi-protocol monitors NETWORK_AVAILABILITY_MONITOR, private locations SYNTHETIC_LOCATION. This
// corrects an earlier note here that claimed no Smartscape equivalent existed. Unlike ActiveGate,
// `fetch dt.entity.synthetic_test` does still work and remains a genuine fallback — it reads the
// classic entity store, which can retain entities Smartscape (live topology) no longer lists.
```

```
// Verify synthetic execution results are being recorded
// Execution results live in dt.synthetic.events, not in the generic events table
fetch dt.synthetic.events, from:-1h
| filter in(event.type, {"http_monitor_execution", "browser_monitor_execution",
"multiprotocol_monitor_execution"})
| summarize executionCount = count()
| fieldsAdd status = if(executionCount &gt; 0, then: "Synthetic monitors executing", else: "No synthetic
results — check monitor configuration")
```

8. Step Completion Checklist

Do not proceed to Step 7 (Enable) until all items are confirmed.

Checkpoint	Status
All dashboard tiles showing data	[ ]
Dashboard ownership and internal links updated to SaaS URLs	[ ]
Hardcoded entity IDs re-pointed to new IDs (or replaced with tag / management-zone filters)	[ ]
All notification channels tested end-to-end (email, Slack, Teams, PagerDuty, ServiceNow)	[ ]
Problem notification resolution events verified	[ ]
CI/CD pipeline integrations updated with SaaS URLs and new tokens	[ ]
Deployment events appearing in SaaS after CI/CD trigger	[ ]
ServiceNow integration reconnected and verified	[ ]
Jira integration updated (if applicable)	[ ]
CMDB synchronization pointing to SaaS APIs	[ ]
Extensions 1.0 rebuilt as Extensions 2.0 (or confirmed SaaS-compatible)	[ ]
Extensions 2.0 deployed on host-based ActiveGates	[ ]
Cloud integrations (AWS, Azure, GCP) reconfigured and metrics flowing	[ ]
All API scripts updated with SaaS URLs and new tokens	[ ]
Meta-monitoring health checks updated for SaaS architecture	[ ]
Synthetic monitors recreated and executing	[ ]
Synthetic private locations deployed (if applicable)	[ ]

Next Step

> **M2S-07: Step 7 — Enable** — Train users and communicate the migration to the organization. Deliver persona-based training, update runbooks and documentation, publish self-service resources, and establish support channels for the broader user base.

Additional Resources

- SaaS Upgrade Assistant Documentation
- Automations (Workflows) Documentation
- Extensions 2.0 Documentation
- Cloud Integrations Overview
- Synthetic Monitoring Documentation
- Account Management API

Summary

In Step 6, you:

- Validated dashboard ownership, internal links, and entity references in SaaS
- Reconnected and tested all alerting notification channels end-to-end
- Updated CI/CD pipeline integrations with SaaS URLs and new API tokens
- Reconfigured ITSM integrations (ServiceNow, Jira, CMDB synchronization)
- Migrated extensions from EF1 to EF2 and reconfigured cloud integrations
- Updated all API scripts and meta-monitoring health checks for SaaS
- Recreated synthetic monitors with appropriate private and public locations

> **Key Takeaway:** Integration is where the migration becomes real for everyone outside the Dynatrace admin team. Developers see deployment events, operations see alerts, and management sees dashboards. Test every channel before declaring Step 6 complete.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.